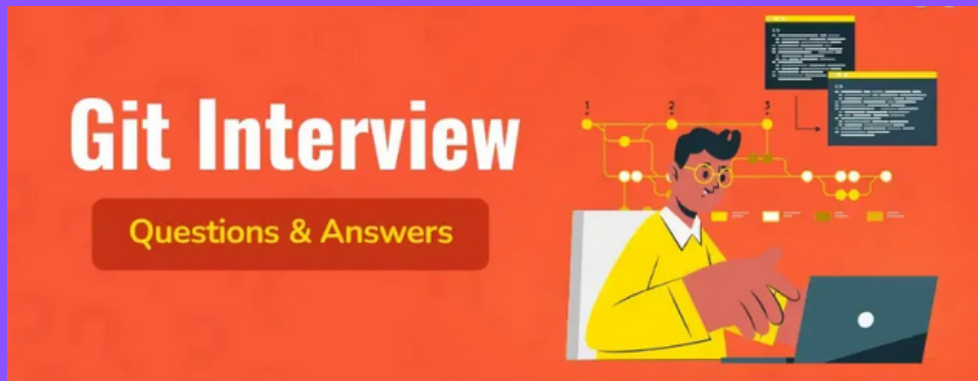




GIT Interview Questions DAY 4

Advanced GIT Interview Questions

30.Which Git command can you use to list all branches that have been merged into master?

- The git branch --merged command lists all branches that have already been merged into the currently checked-out branch.
- It's useful for identifying branches that can be safely deleted.
- Conversely, git branch --no-merged shows branches that have not yet been merged into the current branch, helping you track pending work.

31.How do you identify and resolve merge conflicts in Git?

How Git Merge Conflicts Occur and How to Resolve Them

Merge conflicts in Git typically occur when multiple contributors modify the same lines in a file across different branches. Git is unable to automatically determine which change should take precedence, so it flags a conflict for manual resolution.

📖 Steps to Resolve Merge Conflicts in Git:

1. Identify Conflicted Files

Git highlights conflicted files during a merge and marks them in the working directory. You can also run:

```
git status
```

to see which files are in conflict.

2. Manually Review and Edit Conflicts

Open the conflicted files and look for conflict markers (<<<<<<, =====, >>>>>>). Discuss with your team if necessary and modify the files to include the correct version.

3. Stage the Resolved Files

Once the conflicts are resolved, add the updated files to the staging area:

```
git add <filename>
```

4. Commit the Merge

After staging the changes, complete the merge by committing:

```
git commit
```

5. Push the Changes (if needed)

Push the resolved changes to the remote repository:

```
git push
```

💡 Tip:

Use tools like git mergetool or visual merge tools (e.g., VS Code, Meld, Beyond Compare) for easier conflict resolution.

32. When dealing with a broken commit, is it better to create a new commit or amend the existing one? Why?

While `git commit --amend` can be useful for small corrections, it's generally recommended to create a new commit instead of amending an existing one, especially in shared or production-ready branches. Here's why:

- Preserves Commit History:

Amending a commit rewrites history and destroys the original state of that commit. If only the commit message is being changed, it may be harmless, but altering the actual content can result in loss of valuable information.

- Risk of Unintended Overwrites:

Frequent use of `--amend` can lead to the gradual accumulation of unrelated changes in a single commit, making it harder to trace specific modifications later.

- Best Practice for Traceability:

Creating a new commit ensures a clear, chronological trail of changes, which is easier to audit, debug, and collaborate on.

33. What is the best way to revert a bad commit that has already been pushed to a remote repository?

There are situations where you may need to undo a pushed commit and restore the project to a stable state. Depending on the context, there are two safe and effective approaches:

✅ Approach 1: Fix and Commit Again (Recommended)

If the bad commit introduced issues that can be corrected with additional changes:

- Manually fix the problems in the affected files.
- Create a new commit with the necessary corrections.
- Push the updated commit to the remote repository.

```
git add <fixed files>
```

```
git commit -m "Fix: corrected issues from previous commit"
```

```
git push
```

This is the simplest and most recommended approach, especially in shared branches, as it maintains history and clarity.

✅ Approach 2: Revert the Bad Commit

If you want to undo the effects of a specific commit without editing the files manually:

- Use the `git revert` command to create a new commit that reverses the changes introduced by the bad one.

```
git revert <bad_commit_hash>
```

```
git push
```

This approach is also safe for collaborative environments because it doesn't rewrite history—it simply adds a new commit that undoes the previous one.

34.What does the git cherry-pick command do, and when would you use it?

What Does git cherry-pick Do?

The git cherry-pick command is used to apply specific commits from one branch onto another within the same repository.

A common use case is when you want to backport or forward-port changes, such as applying a bug fix from a maintenance branch to a development branch, without merging the entire branch.

This allows you to selectively bring in changes without affecting unrelated commits.

When to Use git cherry-pick

You would use git cherry-pick when you want to selectively apply specific commits from one branch onto another without merging the full branch history.

Common Scenarios:

1. Backporting a Hotfix:
 - A bug fix was committed to the main or develop branch, and you want to apply that exact fix to a release or maintenance branch.
2. Forward-porting Changes:
 - A fix made in a maintenance branch (e.g., release-1.0) needs to be brought into the active development branch (main or dev).
3. Avoiding Full Merge Overhead:
 - You want just a few specific changes from a feature branch without merging everything it includes.
4. Applying Commits from an Abandoned Branch:
 - A developer started work on a branch but abandoned it; instead of merging everything, you pick only the useful commits.

35.How can you remove a file from the Git index without deleting it from the local filesystem? What are the steps involved?

How to Remove a File from Git Index Without Deleting It Locally

Sometimes, files are unintentionally added to the Git index—often due to an overly broad git add command. If you use git rm, it will remove the file from both the index and your working directory, which may not be what you want.

Instead, you can use git reset to unstage the file (remove it from the index) while keeping it intact in your local file system.

```
git reset <file_name> # Removes the file from the staging area
```

To prevent the file from being accidentally added again in the future, it's a good practice to add it to the .gitignore file:

```
echo <file_name> >> .gitignore # Adds the file to .gitignore
```

This approach keeps your working directory safe while ensuring Git doesn't track the file going forward.

36.What factors should be considered when deciding between using git merge and git rebase?

Choosing Between git merge and git rebase

Both git merge and git rebase are used to integrate changes from one branch into another. However, they operate differently and have distinct implications for your project's history.

- git merge preserves the complete history by creating a new merge commit.
- git rebase rewrites history by moving your commits to a new base, resulting in a cleaner, linear commit history.

When in doubt, it's generally safer to use git merge. Below are key factors to consider when choosing between the two:

When to Prefer git merge:

- Collaboration with others: In shared branches (especially in public or open-source repositories), merging is safer. Rebasing in such cases can lead to conflicts and inconsistencies because it rewrites history others may rely on.
- Preserving historical context: Merge commits retain the context of how and when changes were integrated, which can be useful for tracing features or bug fixes.
- Need for revert-friendly history: Merged commits are easier to revert. Rebasing, on the other hand, may destroy original commit hashes, making it difficult or even impossible to undo safely.

When to Use git rebase (with caution):

- Private or individual branches: If you're working alone or your branch is not yet shared with others, rebasing can help keep a clean, linear history.
- Branches per developer model: When branches represent individuals (rather than features), rebasing can be used safely since there's less historical context to preserve.
- Before merging to main branches: Some teams prefer to rebase a feature branch onto the latest main before merging, to reduce merge commits and simplify history.

Important Notes:

- Rebasing rewrites commit history, which can lead to data loss if not used carefully.
- Avoid rebasing public or shared branches unless you're using `git pull --rebase` in a controlled environment.
- Always ensure you understand the implications of rebasing before using it, especially in collaborative projects.

37.How would you identify the commit that introduced a bug after a merge operation?

Identifying a Bug-Introducing Commit with git bisect

When you're unsure where a bug was introduced—especially after several commits or a merge—manually inspecting changes can be slow and tedious. Thankfully, Git provides an efficient solution using binary search with the `git bisect` command.

Step-by-Step Process:

1. Start the bisecting session
`git bisect start`
2. Mark the current (buggy) commit as bad
`git bisect bad`
3. Mark a known good commit (where everything worked fine)
`git bisect good <commit-hash>`

At this point, Git automatically checks out a commit halfway between the good and bad commits. You test the code and mark the result:

- If it's good:
git bisect good
- If it's bad:
git bisect bad

Git continues narrowing down the search range by checking out the midpoint of the remaining commits. This process repeats until the exact commit that introduced the issue is found.

git bisect is a powerful debugging tool that dramatically reduces the effort needed to locate problematic changes in large or complex commit histories.

38.What is the purpose of git reset --mixed and git merge --abort, and in what scenarios would you use them?

The git reset --mixed command is used to undo changes in the Git index (staging area) while keeping the changes in the working directory intact. It resets the index to match a specific commit but does not delete any uncommitted changes from the files themselves.

On the other hand, the git merge --abort command is used to cancel an ongoing merge process and restore the repository to the exact state it was in before the merge began. This is particularly helpful when conflicts arise during a merge and you decide not to proceed.

39.Can you explain the difference between git revert and git reset, and when to use each?

 git revert vs. git reset; Key Differences

- git revert is used to create a new commit that undoes the effects of a previous commit. It preserves the repository's history by appending a new commit rather than modifying existing ones.
- git reset is used to undo local changes by moving the current branch pointer to a different commit. It can modify the commit history, the staging area (index), and the working directory, depending on the reset mode (--soft, --mixed, or --hard).

Feature	git revert	git reset
Purpose	Reverts a specific commit by creating a new one	Moves HEAD to a specified commit
History	Preserved (safe for shared repos)	Can rewrite history (use with caution)
Impact	Adds new commit	Affects commit history, index, and working directory
Use Case	Undo specific commits on shared branches	Remove or change local commit history